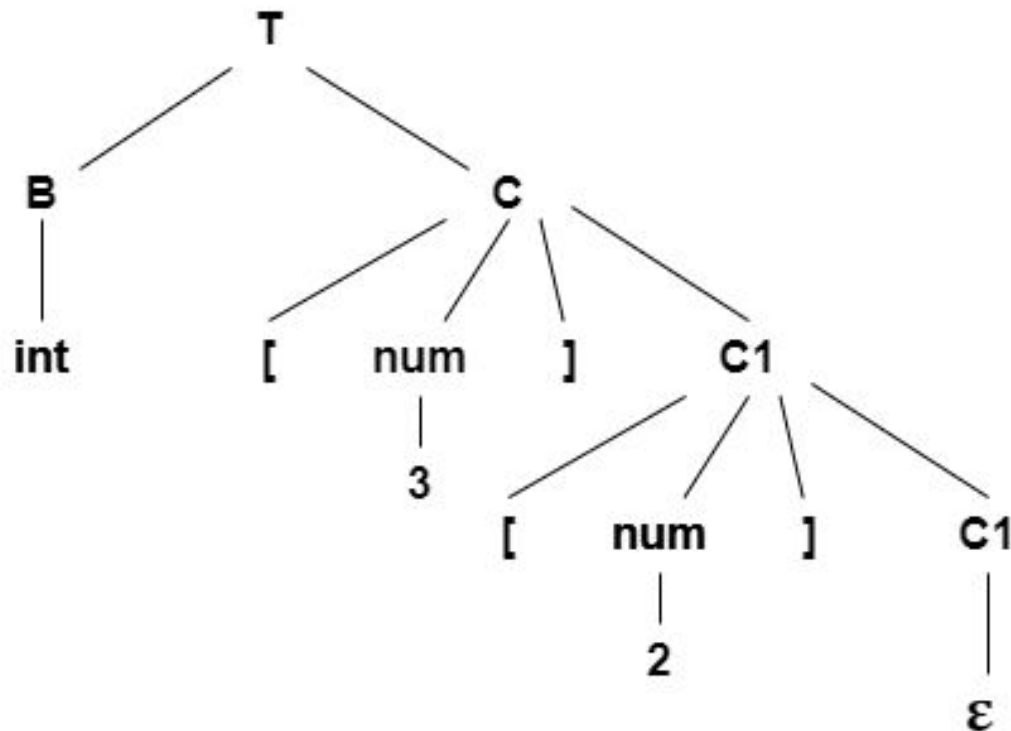


Production	Semantic Rules
T $\rightarrow$ BC	T.t = C.t
	C.b = B.t
B $\rightarrow$ int	B.t = integer
B $\rightarrow$ float	B.t = float
C $\rightarrow$ [num] C1	C.t = array(num.val, C1.t)
	C1.b = C.b
C $\rightarrow \epsilon$	C.t = C1.b
	C1.b = C.b

- Nonterminal T generates either a basic type or an array type.
- T generates a basic type when T derives B C and C derives ϵ
- Nonterminal B generates one of the basic types int and float.
- Otherwise, C generates array components consisting of a sequence of integers, each integer surrounded by brackets.
- The nonterminals B and T have a synthesized attribute t representing a type.
- The nonterminal C has two attributes: an inherited attribute b and a synthesized attribute t
- The inherited b attributes pass a basic type down the tree, and the synthesized t attributes accumulate the result

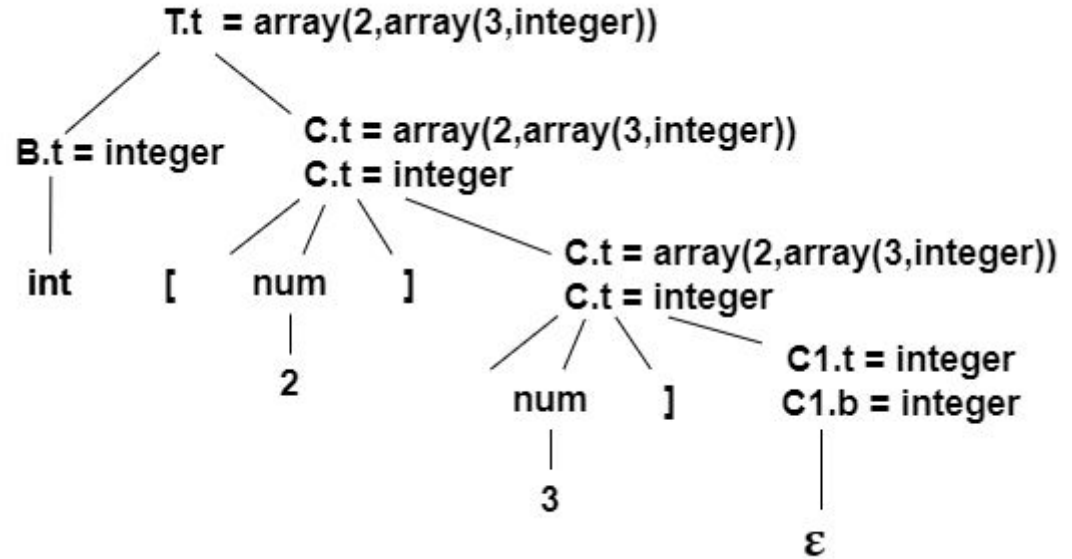
Production	Semantic Rules
T $\rightarrow$ BC	T.t = C.t
	C.b = B.t
B $\rightarrow$ int	B.t = integer
B $\rightarrow$ float	B.t = float
C $\rightarrow$ [num] C1	C.t = array(num.val, C1.t)
	C1.b = C.b
C $\rightarrow \epsilon$	C.t = C1.b
	C1.b = C.b



Annotated Parse tree for int [2] [3]

Prepared By: Sukarna Sarker

Production	Semantic Rules
T -> BC	T.t = C.t
	C.b = B.t
B -> int	B.t = integer
B -> float	B.t = float
C -> [num] C1	C.t = array(num.val, C1.t)
	C1.b = C.b
C -> ε	C.t = C1.b
	C1.b = C.b



Syntax-directed translation of array types

Type expression: array(2, array(3, integer))

Prepared By: Sukarna Sarker

Production	Semantic Rules
T $\rightarrow$ BC	T.t = C.t
	C.b = B.t
B $\rightarrow$ int	B.t = integer
B $\rightarrow$ float	B.t = float
C $\rightarrow$ [num] C1	C.t = array(num.val,C1.t)
	C1.b = C.b
C $\rightarrow \epsilon$	C.t = C1.b
	C1.b = C.b

Note that, if types are represented by trees, then this operator returns a tree node labeled array with two children for a number and a type.

